

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсовой работы
ассистент кафедры информатики
_____ А.Г. Бокун
____.____.2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
курсового проекта
на тему:
«ПРИЛОЖЕНИЕ ДЛЯ УЧЕТА ЛИЧНЫХ ФИНАНСОВ»

БГУИР КП 6-05-0612-02 01 ПЗ

Выполнил студент группы 453503
АВРАМЕНКО Роман
Александрович

(подпись студента)

Курсовая работа представлена на
проверку _____.____.2026

(подпись студента)

Минск 2026

ВВЕДЕНИЕ

В современных условиях цифровой экономики и стремительного развития финансовых технологий вопросы рационального управления личными денежными средствами приобретают особую актуальность. Увеличение количества безналичных операций, использование банковских карт, онлайн-платежей и электронных сервисов приводит к тому, что человек ежедневно совершает множество финансовых транзакций, которые трудно контролировать без специальных инструментов. Отсутствие систематизированного учета доходов и расходов нередко становится причиной неэффективного распределения бюджета, возникновения задолженностей и невозможности достижения долгосрочных финансовых целей.

Значительная часть людей либо вовсе не ведет учет личных финансов, либо делает это нерегулярно, например, используя записи в блокноте или в электронных таблицах. Такой подход не обеспечивает целостного представления о финансовом состоянии, не позволяет быстро анализировать структуру расходов и прогнозировать будущие траты. В результате снижается финансовая дисциплина и повышается риск необоснованных расходов.

Одним из решений данной проблемы является создание специализированной системы учета личных финансов, предназначенной для регистрации, хранения и анализа данных о доходах, расходах и балансе пользователя. Система учета личных финансов обеспечивает четкую структуру финансовых операций и их разделение по категориям, периодам и источникам, а также предоставляет инструменты для анализа и визуализации данных.

Основное назначение такой системы заключается в повышении прозрачности личного бюджета, формировании осознанного отношения к расходам и поддержке процесса финансового планирования. Приложение позволяет пользователю:

- фиксировать личные доходы и расходы;
- распределять финансовые операции по категориям;
- формировать понятные и разносторонние отчеты за выбранный период;
- быстро и удобно анализировать динамику изменения бюджета;
- планировать накопления и контролировать достижение финансовых целей.

Целью данного курсового проекта является разработка приложения для учета личных финансов с использованием принципов объектно-ориентированного программирования, обеспечивающего ввод, хранение и анализ финансовых данных пользователя.

Для достижения этой цели были поставлены следующие задачи:

- провести анализ предметной области и изучить существующие аналоги;
- определить основные функциональные требования к системе;
- разработать объектную модель приложения, выделив ключевые классы и их взаимосвязи;
- реализовать основные функциональные возможности;
- обеспечить хранение данных и их корректную обработку;
- реализовать механизм формирования отчетов и анализа расходов по категориям;
- провести тестирование разработанного программного продукта.

В ходе выполнения курсового проекта были изучены литературные источники, которые позволили сформировать теоретическую основу работы, определить принципы построения архитектуры приложения и обосновать актуальность разработки системы учета личных финансов.

1 АНАЛИЗ СУЩЕСТВУЮЩИХ АНАЛОГОВ

Перед разработкой программного продукта целесообразно провести анализ уже существующих решений в рассматриваемой предметной области. Исследование аналогичных программ позволяет определить текущий уровень развития систем учёта личных финансов, выявить их функциональные возможности, сильные и слабые стороны, а также определить важные направления при разработке приложения.

На современном рынке программного обеспечения представлено значительное количество приложений для учета личных финансов. Они различаются по функциональности, архитектуре, способу хранения данных, наличию синхронизации, уровню аналитических возможностей и удобству пользовательского интерфейса. Одни приложения ориентированы на базовый учет доходов и расходов, другие предлагают расширенные инструменты финансового планирования, работу с несколькими валютами и построение детализированной отчетности.

В ходе выполнения курсового проекта были рассмотрены два приложения для учёта личных финансов. Для каждого из них был проведён анализ общей информации о продукте, основных функциональных возможностей, а также преимуществ и недостатков с точки зрения удобства использования и полноты реализованных функций.

Результаты анализа были использованы при формировании требований к разрабатываемому приложению и обосновании выбора реализуемого функционала.

1.1 Приложение «Домашняя бухгалтерия»

«Домашняя бухгалтерия» (рисунок 1.1) – это приложение для учёта личных финансов, предназначенное для пользователей без глубоких бухгалтерских знаний. Разработка изначально ориентирована на русскоязычный рынок и получила достаточно широкое распространение среди пользователей благодаря простому интерфейсу и базовому набору функций для ведения бюджета.

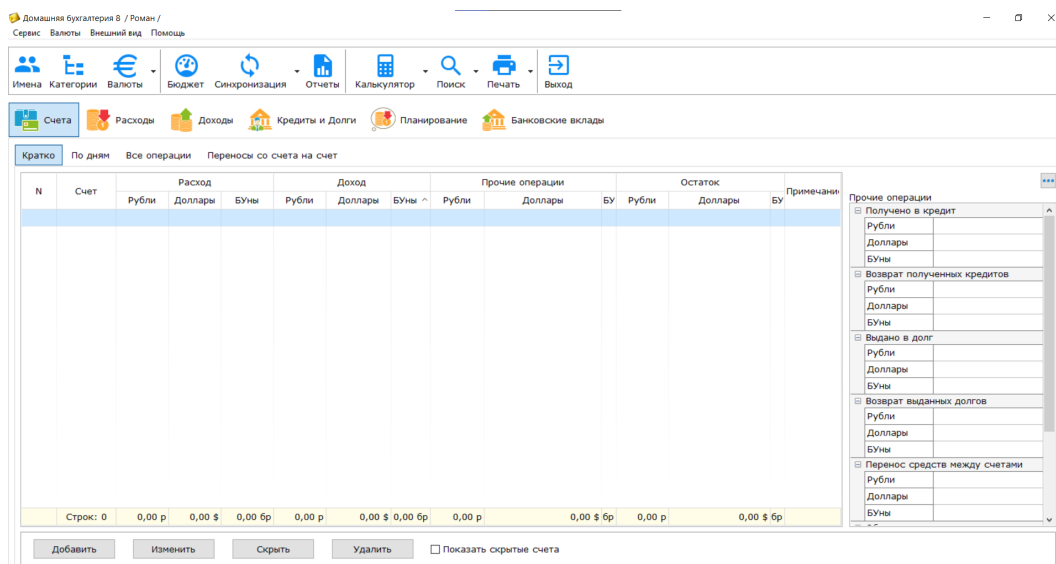


Рисунок 1.1 – Интерфейс приложения «Домашняя бухгалтерия»

Программа предоставляет возможность создавать несколько аккаунтов, задавать пароль и восстанавливать его по ключевому слову, учитывать доходы и расходы, классифицировать операции по категориям, следить за личными вкладами и кредитами, планировать бюджет и анализировать финансовые данные за выбранные периоды, хранить финансы в различной валюте и проводить переводы из одной валюты в другую. При каждом выходе из приложения создается резервная копия данных на персональном компьютере пользователя, позволяющая восстановить их в случае утери или порчи.

Приложение распространяется на платной основе с бесплатным тридцатидневным пробным периодом.

Основные достоинства приложения «Домашняя бухгалтерия»:

- добавление различных валют и их актуализация;
- возможность добавления единиц измерения (килограммы, десятки, упаковки и другие);
- быстрый доступ к калькулятору для произведения предварительных расчетов;
- функция поиска;
- синхронизация с мобильным приложением.

Основной недостаток приложения «Домашняя бухгалтерия» – это пользовательский интерфейс. Практически все данные о транзакциях представлена в виде таблиц, что ухудшает наглядность информации по сравнению с графическим отображением финансовых операций в виде графиков и диаграмм.

1.2 Приложение «Alzex Finance Pro»

«Alzex Finance Pro» (рисунок 1.2) – это настольное приложение для управления личными и семейными финансами. Программа ориентирована на

пользователей, которым требуется наглядный и функциональный инструмент для учета доходов, расходов, бюджета и финансового планирования. Приложение предлагается на нескольких платформах, включая *Windows*, *macOS*, *iOS* и *Android*, с возможностью синхронизации данных между устройствами через облачные сервисы. Такой мультиплатформенный подход повышает удобство использования, поскольку пользователи могут вести учет финансов как на персональном компьютере, так и на мобильных устройствах.

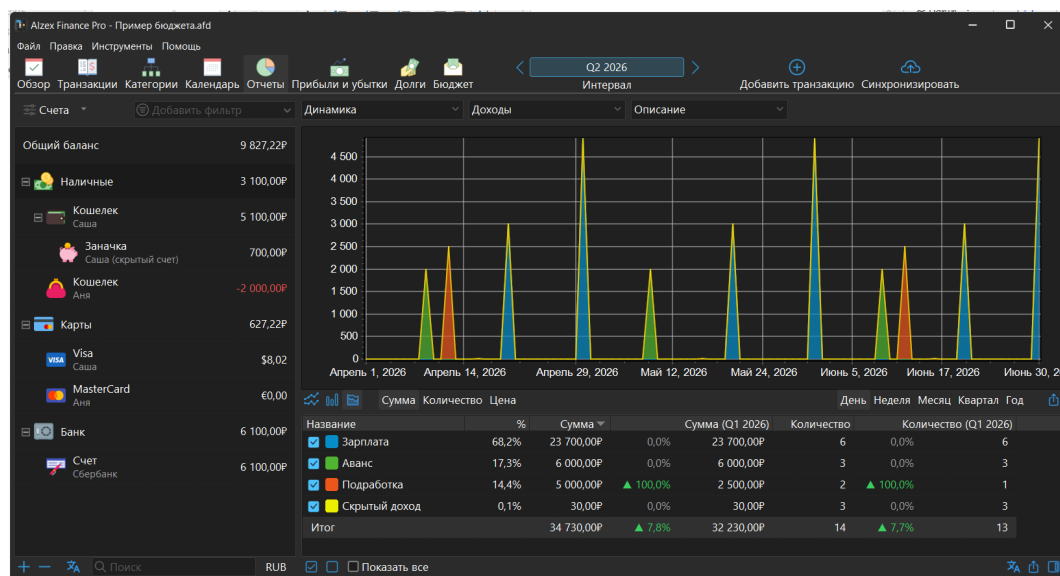


Рисунок 1.2 – Интерфейс приложения «*Alzex Finance Pro*»

«*Alzex Finance Pro*» отличается интуитивно понятным графическим интерфейсом и достаточным набором функций, которые позволяют гибко организовать учёт средств, анализировать финансовую активность и планировать бюджет. Кроме того, приложение поддерживает работу с несколькими членами семьи с настройкой доступа, что делает его не только инструментом личного учета, но и средством совместного управления семейным бюджетом.

«*Alzex Finance Pro*» реализует возможность построение детальных и разноплановых динамических отчетов за определенный период (день, месяц, квартал, год или произвольный промежуток времени), а также позволяет открывать отдельные счета для накоплений, группировать счета по категориям и по принадлежности к конкретному члену семьи, добавлять теги для более удобного составления отчетов. Также в приложении поддерживается возможность отслеживания долгов, кредитов и счетов в банке с автоматическим расчетом периодических выплат и напоминаниями. Доступна синхронизация нескольких устройствах.

При первом запуске программы пользователю предоставляется возможность потренироваться в использовании функций приложения на уже

сформированной базе данных. Это положительно сказывается на скорости адаптации к функционалу приложения.

Программа распространяется на платной основе с различными тарифами и пробной версией с возможностью добавления до двух сотен записей.

Основные достоинства:

- разнообразное графическое представление данных;
- наличие широкого спектра валют (в том числе криптовалют);
- календарь с подробной информацией о запланированных тратах и поступлениях;
- удобное переключение между промежутками времени.

Недостатком приложения «*Alzex Finance Pro*» является отсутствие некоторых функций: хранение финансов в различной валюте, поиск транзакция по различным параметрам, добавление единиц измерения и локальное резервное копирование.

Проведённый сравнительный анализ показал, что на рынке программного обеспечения представлен широкий спектр инструментов для управления личными финансами, однако ни одно из рассмотренных решений не является идеальным. «Домашняя бухгалтерия» обладает богатым функционалом, но проигрывает в наглядности и эргономике интерфейса, перегружая пользователя табличными данными. В свою очередь, «*Alzex Finance Pro*» предлагает современный визуальный подход, но имеет ограничения в гибкости настройки специфических функций и доступе к глубокой кастомизации.

На основании проведённого анализа можно сделать вывод о целесообразности разработки собственного приложения для учёта личных финансов. Разрабатываемый программный продукт должен устранить выявленные недостатки, объединив в себе преимущества рассмотренных решений:

1 Сбалансированный набор инструментов, сфокусированный на ключевых потребностях пользователя: категоризации операций, прозрачной аналитике и удобном планировании бюджета.

2 Интуитивно понятный графический интерфейс, обеспечивающий высокую наглядность финансовых данных.

На основании проведенного анализа можно сделать вывод, что создание данного приложения является актуальной задачей, так как позволит реализовать систему, сочетающую техническую чистоту кода с удобством для конечного пользователя.

2 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ

На основе анализа предметной области и изучения аналогов был сформирован перечень требований к разрабатываемому приложению. Система должна обеспечивать полный цикл управления личными финансами, поддерживая гибкую структуру данных и аналитические возможности.

2.1 Функциональные требования

2.1.1 Управление пользователями и разграничение доступа:

- идентификация и аутентификация: реализация системы входа в персональный аккаунт по уникальному логину и паролю;
- многопользовательский режим: возможность регистрации нескольких независимых учетных записей в рамках одной системы;
- изоляция данных: каждый пользователь обладает уникальным пространством данных (счета, транзакции, категории, бюджеты).

2.1.2 Организация структуры финансовых инструментов (счета):

- управление жизненным циклом счетов: реализация функций создания, редактирования (изменение названия, иконки, типа) и архивирования/удаления счетов;
- древовидная структура: возможность объединять счета в группы (например, группа «Банки» может включать счета «Дебетовая карта» и «Кредитка») и автоматическое агрегирование баланса, то есть баланс родительского счета должен автоматически рассчитываться как сумма остатков всех его дочерних подсчетов;
- механизм межбалансовых переводов: реализация функционала перемещения средств между счетами пользователя.

2.1.3 Классификация и группировка данных:

- категории: поддержка древовидной структуры (основные и дочерние категории) для детализации расходов и доходов;
- члены семьи: возможность привязки транзакций к конкретным участникам бюджета для анализа персональных трат;
- теги: использование меток для дополнительной фильтрации транзакций, не привязанной к основной иерархии категорий.

2.1.4 Управление транзакциями и финансовыми операциями:

- регистрация операционной деятельности: возможность создания записей о доходах и расходах;
- поддержка расширенной атрибуции каждой операции: сумма, дата и точное время, счет списания/зачисления, категории, привязка к члену семьи, набор текстовых тегов;

- сортировка: упорядочивание списка операций по атрибутам и их группировка;

- поддержка планируемых транзакций: создание шаблонов операций, которые должны произойти в будущем, с механизмом уведомления и подтверждения пользователем в указанную дату.

2.1.5 Управление кредитными инструментами и долговыми обязательствами:

- учет кредитов: регистрация кредитов с указанием общей суммы, процентной ставки, срока кредитования и даты оформления;

- гибкие модели начисления процентов: реализация разных подходов к начислению процентов на первоначальную сумму, в том числе простые и сложные проценты и единоразовые начисления;

- сортировка: упорядочивание списка операций по атрибутам и их группировка;

- генерация графика платежей: автоматический расчет ежемесячного взноса на весь срок кредита, а также автоматическое создание планируемых транзакций на основе графика выплат.

2.1.6 Создание бюджетов и контроль:

- система лимитов: установка финансовых ограничений на определенные периоды;

- фильтрация транзакций: настройка гибких фильтров для бюджетов (ограничение трат по конкретным категориям, тегам или счетам);

- отображение результатов: визуальное отслеживание остатка бюджета и информирование о превышении лимитов.

2.2 Нефункциональные требования

Помимо реализации основных функций, к программному продукту предъявляется ряд технических требований, обеспечивающих качество архитектуры, стабильность работы и удобство дальнейшего сопровождения кода. Основные нефункциональные требования:

- архитектурная чистота: построение системы в соответствии с принципами *Clean Architecture* и *SOLID*, а также использование паттернов проектирования для обеспечения гибкости бизнес-логики;

- надежность и целостность данных: корректная обработка исключительных ситуаций и обеспечение транзакционности при выполнении операций, влияющих на баланс счетов;

- масштабируемость: возможность расширения функционала, добавления новых типов отчетов или методов расчета без изменения существующего ядра системы;

– эргономичность: интуитивно понятный интерфейс, обеспечивающий быстрый доступ к основным функциям и корректную обратную связь при ошибках ввода;

– производительность: оптимизация работы с базой данных для быстрого формирования отчетов и фильтрации больших списков транзакций.

3 ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОЕКТИРОВАНИЕ

3.1 Основные понятия объектно-ориентированного программирования

Объектно-ориентированное проектирование (ООП) представляет собой методологию, основанную на представлении программной системы в виде совокупности объектов, каждый из которых является экземпляром определенного класса, а классы образуют иерархию наследования. В контексте разработки системы учета личных финансов данный подход позволяет эффективно моделировать объекты реального мира (счета, транзакции, бюджеты) и определять их поведение.

3.1.1 Инкапсуляция в рамках разрабатываемой системы выполняет две ключевые функции: ограничение прямого доступа к состоянию объекта и объединение данных с методами их обработки. В отличие от традиционного процедурного подхода, где данные и функции для их обработки существуют отдельно, инкапсуляция позволяет связать их внутри единой сущности. Например, объект *Account* в приложении не просто хранит информацию о своем названии или типе, но и наделяется поведением, например, как рассчитать свой текущий баланс. Для внешнего программного кода процесс этого вычисления остается скрытым: пользователь объекта лишь обращается к свойству баланса, не вникая в алгоритм его формирования.

Такое скрытие внутренней реализации особенно критично при работе с иерархическими структурами. Внешние компоненты системы, запрашивающие состояние счета, не зависят от того, является ли данный объект «дочерним» или «родительским». Для счета инкапсулированная логика заключается в суммировании всех связанных с ним транзакций, в то время как для родительской группы счетов поведение метода будет иным – объект рекурсивно опрашивает свои дочерние элементы и агрегирует их показатели.

Дополнительно инкапсуляция обеспечивает целостность финансового состояния системы. Все изменения данных, такие как проведение перевода или корректировка операции, происходят исключительно через строго определенные методы класса. Благодаря тому, что эти правила сосредоточены внутри самого класса, исключается риск появления некорректных данных из-за ошибок во внешних модулях приложения.

3.1.2 Наследование в рамках проектирования системы выступает механизмом, позволяющим создавать иерархические связи между классами и повторно использовать программный код, минимизируя его дублирование. Данная концепция предполагает создание базового класса, содержащего общие свойства и методы, и производных классов, которые наследуют этот функционал и дополняют его специфической логикой.

Наследование стало основой для реализации математического аппарата системы, в частности – механизмов начисления процентов по долговым обязательствам. Для решения этой задачи был спроектирован базовый класс для расчета процентных начислений, от которого наследуется семейство похожих специализированных классов. Каждый из них переопределяет логику вычислений в соответствии с выбранной финансовой моделью: единоразовым начислением, простыми или сложными процентами. Такой подход гарантирует, что все классы-калькуляторы имеют схожую структуру и предсказуемое поведение, что значительно упрощает их интеграцию в общую логику кредитного модуля и обеспечивает высокую расширяемость системы при необходимости добавления новых банковских продуктов.

3.1.3 Полиморфизм представляет собой концепцию, позволяющую использовать единый программный интерфейс для взаимодействия с объектами различных типов, скрывая за общим вызовом специфическую внутреннюю реализацию. В контексте разрабатываемой системы полиморфизм обеспечивает гибкость и адаптивность архитектуры, позволяя обрабатывать разнообразные сущности единообразным способом без необходимости написания уникального кода под каждый частный случай.

Наиболее ярко данный принцип реализован в модуле управления бюджетами при создании системы ограничений на транзакции. Для контроля лимитов был спроектирован общий механизм фильтрации, который позволяет накладывать ограничения на финансовые операции по самым разным критериям: категориям, тегам или конкретным счетам. Благодаря полиморфизму, система может взаимодействовать с любым типом ограничения через абстрактный интерфейс. Например, при проверке транзакции на соответствие бюджету приложение вызывает универсальный метод проверки, который ведет себя по-разному в зависимости от того, какой именно объект ограничения используется в данный момент. Это позволяет комбинировать различные правила контроля, не усложняя основную логику приложения.

3.2 Применение принципов *SOLID*

Эффективное применение объектно-ориентированного подхода требует соблюдения определенных архитектурных стандартов, которые гарантируют гибкость, читаемость и легкость сопровождения программного продукта. Одним из наиболее признанных наборов таких правил является акроним *SOLID* — пять основополагающих принципов проектирования, направленных на снижение связанности компонентов и повышение надежности системы.

3.2.1 Принцип единственной ответственности (*Single Responsibility Principle*) утверждает, что каждый класс или модуль должен отвечать за одну-единственную часть функциональности системы, и у него должна быть лишь одна причина для изменения. В разрабатываемом приложении этот принцип

стал основой для разделения бизнес-логики на независимые сервисы, каждый из которых изолирует конкретный аспект обработки данных.

В рамках этой архитектуры сервис транзакций (*Transaction Service*) отвечает исключительно за регистрацию и валидацию финансовых операций, обеспечивая корректное создание записей, проверку баланса счетов. Параллельно с ним сервис бюджетов (*Budget Service*) инкапсулирует логику контроля лимитов, занимаясь анализом потока транзакций, их сопоставлением с установленными ограничениями и предоставлением данных о текущем состоянии бюджета. За управление кредитными продуктами отвечает специализированный сервис долгов (*Debt Service*), чья деятельность сосредоточена на обновлении состояния обязательств на определенную дату, расчете начисленных процентов и автоматической генерации плановых платежей согласно графику.

Такое разделение позволяет модифицировать алгоритмы работы одного модуля, например, правила контроля бюджетов, не рискуя нарушить стабильность работы механизмов регистрации транзакций или расчёта кредитов. Подобная декомпозиция значительно упрощает тестирование системы, так как каждый сервис можно проверять в изоляции, что исключает появление скрытых побочных эффектов и повышает общую надежность программного продукта.

3.2.2 Принцип открытости/закрытости (*Open-Closed Principle*) постулирует, что программные сущности должны быть открыты для расширения, но закрыты для модификации. Это означает, что при необходимости изменения поведения системы или добавления новой функциональности разработчик не должен вносить правки в уже написанный и протестированный код, а должен иметь возможность дополнить систему новыми модулями.

Ярким примером применения этого принципа является механизм расчета процентов по кредитам. Вместо жестко прописанной логики в основном коде приложения был разработан провайдер стратегий расчета (*DebtInterestCalculatorProvider*), который принимает список всех доступных классов-реализаций. Каждая такая реализация (*DebtInterestCalculator*) оформлена в виде отдельного класса, следующего общему интерфейсу. Провайдер автоматически определяет нужную логику в зависимости от типа кредита. Таким образом, для добавления в систему нового банковского продукта с уникальной формулой начисления процентов достаточно создать новый класс и зарегистрировать его в системе, не изменяя при этом код провайдера или сервиса управления долгами. Аналогичный подход применен и в системе ограничений бюджета. Провайдер правил фильтрации позволяет легко расширять критерии, по которым транзакции попадают под действие бюджетного лимита.

3.2.3 Принцип подстановки Барбары Лисков (*Liskov Substitution*

Principle) утверждает, что объекты в программе должны быть заменяемы экземплярами их подтипов без изменения правильности работы программы. Другими словами, производный класс не должен нарушать контракты, установленные базовым классом или интерфейсом.

В качестве примера реализации данного принципа можно привести управление доступом к репозиториям базы данных. Система спроектирована таким образом, что основной сервис, работающий с данными, ожидает объект, реализующий стандартный интерфейс репозитория. Однако вместо прямой работы с базой данных система подставляет прокси-объект, который берет на себя функции ограничения параллельного доступа и управления очередью запросов. Поскольку прокси-объект строго следует интерфейсу базового репозитория и сохраняет его исходное поведение, остальная часть приложения продолжает работать корректно, не замечая подмены.

3.2.4 Принцип разделения интерфейса (*Interface Segregation Principle*) постулирует, что программные сущности не должны зависеть от методов, которые они не используют. Вместо создания одного универсального интерфейса, объединяющего все возможные действия, предпочтительнее проектировать несколько специализированных интерфейсов, каждый из которых решает узкую задачу. В разрабатываемой системе этот принцип был применен при проектировании уровня доступа к данным.

Вместо реализации единого интерфейса репозитория для всех сущностей, функционал был разделен на несколько узкоспециализированных контрактов. Это позволяет сервисам системы запрашивать ровно тот уровень доступа, который им необходим в данный момент. Так, сервис работы с переводами (*Transfers*) зависит только от интерфейса репозитория переводов (*ITransferRepository*).

Такой подход не только повышает безопасность системы, ограничивая права доступа на уровне архитектуры, но и делает код более понятным и тестируемым. Разделение интерфейсов репозитория гарантирует, что классы-клиенты остаются слабо связанными с реализацией механизмов хранения данных и защищены от изменений в тех методах, которые не имеют отношения к их непосредственной деятельности.

3.2.5 Принцип инверсии зависимостей (*Dependency Inversion Principle*) завершает набор правил *SOLID* и постулирует два важных положения: модули верхних уровней не должны зависеть от модулей нижних уровней (оба должны зависеть от абстракций), а абстракции не должны зависеть от деталей (детали должны зависеть от абстракций).

В качестве практической реализации этого принципа выступает организация взаимодействия между сервисами бизнес-логики и слоем доступа к данным. Высокоуровневые сервисы, например сервис транзакций (*TransactionService*), не имеют прямой зависимости от конкретной реализации

базы данных (*Entity Framework*, *SQL Server* или локальное хранилище). Вместо этого они взаимодействуют исключительно с абстрактными интерфейсами репозитория (*ITransactionRepository*). Сами же конкретные классы репозитория, реализующие работу с базой данных, зависят от этих интерфейсов.

Такой подход позволяет полностью изолировать финансовую логику от инфраструктурных изменений. Поскольку ядро системы зависит от абстракций, замена типа используемой базы данных или переход на внешнюю *API* для хранения информации не требует внесения изменений в программный код бизнес-правил.

3.3 Паттерны проектирования в архитектуре приложения

Паттерны проектирования представляют собой архитектурные решения типовых задач, возникающих в процессе объектно-ориентированного проектирования. Использование шаблонов позволяет не только ускорить процесс разработки, предоставляя готовые схемы взаимодействия объектов, но и существенно повысить качество кода, делая его более предсказуемым, тестируемым и понятным для других разработчиков.

В рамках создания системы учета личных финансов применение паттернов является необходимым инструментом для управления сложностью бизнес-логики. Паттерны обеспечивают единообразие в реализации таких задач, как создание объектов, управление доступом к данным и выбор стратегий расчетов.

В архитектуре приложения паттерны разделены на несколько групп: порождающие (для оптимизации процесса создания сложных сущностей), структурные (для организации связей между компонентами) и поведенческие (для гибкой настройки логики работы системы). Далее будут подробно рассмотрены конкретные шаблоны проектирования, ставшие основой для реализации ключевых модулей программы.

3.3.1 Паттерн Стратегия (*Strategy*) относится к категории поведенческих шаблонов проектирования и предназначен для определения семейства алгоритмов, инкапсуляции каждого из них и обеспечения их взаимозаменяемости. Это позволяет выбирать конкретный алгоритм выполнения задачи непосредственно во время работы программы, не внося изменений в код классов, которые эти алгоритмы используют. В разрабатываемой системе данный паттерн нашел применение в двух критически важных функциональных модулях.

Во-первых, он используется для расчета кредитных обязательств: благодаря созданию общего интерфейса для калькуляторов процентов, каждая конкретная модель — будь то простые проценты, сложные проценты с капитализацией или единовременное начисление — вынесена в отдельный класс-

стратегию. Это позволяет финансовому ядру системы корректно вычислять графики платежей для любого типа кредита, просто подставляя нужную математическую модель в момент проведения расчетов.

Во-вторых, на основе стратегий реализована система гибкой фильтрации бюджетов. Общий интерфейс фильтрации позволяет динамически комбинировать различные условия проверки транзакций, такие как принадлежность к определенным категориям, тегам или конкретным счетам.

Использование данного паттерна обеспечило системе высокую степень гибкости, так как добавление нового типа кредитного продукта или нового критерия фильтрации теперь не требует изменения существующей логики сервисов. Для расширения функционала достаточно создать новый класс, реализующий соответствующий интерфейс.

3.3.2 Паттерн Наблюдатель (*Observer*) относится к категории поведенческих шаблонов проектирования и определяет зависимость между объектами таким образом, что при изменении состояния одного объекта все зависящие от него узлы автоматически получают уведомление. В архитектуре разрабатываемого приложения данный паттерн реализован через специализированный компонент – мессенджер событий, который выступает центральным узлом для обмена оповещениями между различными модулями системы. Использование такого посредника позволяет компонентам системы взаимодействовать друг с другом, не имея прямых ссылок друг на друга, что существенно снижает связанность архитектуры.

В данной модели мессенджер событий аккумулирует оповещения, которые генерируются сервисами управления данными сразу после внесения изменений в базу данных. Компоненты отображения и бизнес-логики выступают в роли подписчиков: они регистрируются в мессенджере и реагируют на конкретные типы событий. Например, при успешном добавлении транзакции сервис отправляет сигнал в мессенджер, после чего модуль истории операций автоматически подгружает новую запись, а виджет общего баланса пересчитывает итоговые суммы.

Важной особенностью реализации является двусторонний характер взаимодействия: любой компонент системы может не только получать уведомления, но и сам отправлять сообщения в мессенджер после завершения своих внутренних процессов. Это обеспечивает реактивность интерфейса и гарантирует, что любое действие пользователя мгновенно отразится во всех связанных разделах программы.

3.3.3 Паттерн Прокси (*Proxy*) относится к структурным шаблонам проектирования и позволяет подставить вместо реального объекта специальный объект-заместитель, который контролирует доступ к оригиналу,

позволяя выполнять дополнительные действия до или после передачи вызова. В разрабатываемой системе данный паттерн был применен для управления доступом к контексту базы данных (*FinansiesDbContext*), что позволило решить проблему конкуренции за ресурсы при выполнении асинхронных запросов.

Поскольку контекст базы данных в используемой *ORM* (*Entity Framework Core*) не является потокобезопасным, одновременное обращение к нему из разных частей приложения может привести к критическим ошибкам. Для предотвращения подобных ситуаций был реализован прокси-слой, который перехватывает обращения к методам репозитория. Этот объект-заместитель инкапсулирует в себе логику разграничения доступа, гарантируя, что в конкретный момент времени только один поток может выполнять операцию записи или чтения. Использование прокси позволило внедрить механизмы очередности прозрачно для остальной бизнес-логики: сервисы продолжают работать с интерфейсом репозитория, не заботясь о внутренних технических сложностях синхронизации потоков.

3.3.5 Паттерн Репозиторий (*Repository*) относится к категории архитектурных шаблонов и выступает в роли посредника между слоем бизнес-логики и слоем доступа к данным. Его основная задача – абстрагировать процесс получения и сохранения объектов, предоставляя остальной системе интерфейс, напоминающий работу с обычной коллекцией объектов в оперативной памяти. В разрабатываемом приложении репозитории позволяют полностью скрыть детали реализации запросов к базе данных, такие как использование *SQL*-команд или специфических методов *Entity Framework*.

Внедрение данного паттерна позволило централизовать логику работы с данными для каждой ключевой сущности: транзакций, счетов и кредитных обязательств. Вместо того чтобы прописывать логику обращения к базе данных внутри сервисов, приложение обращается к соответствующему репозиторию, который берет на себя ответственность за извлечение необходимых записей, их фильтрацию и маппинг в объекты доменной модели. Это обеспечивает чистоту кода бизнес-логики и позволяет легко изменять структуру базы данных или даже менять саму технологию хранения без необходимости переписывания основных алгоритмов приложения.

Кроме того, использование репозитория в сочетании с интерфейсами значительно упрощает процесс модульного тестирования. При тестировании сервисов реальный репозиторий, работающий с физической базой данных, может быть легко заменен на фиктивный объект (*Mock*), возвращающий заранее подготовленные наборы данных.

Завершающим элементом архитектурной модели приложения является паттерн *MVVM (Model-View-ViewModel)*. В то время как рассмотренные выше шаблоны проектирования преимущественно организуют внутреннюю логику и взаимодействие с данными, *MVVM* отвечает за структурирование пользовательского интерфейса и его связь с ядром системы. Детальное описание реализации *MVVM*, механизмов привязки данных и организации взаимодействия между *View* и *ViewModel* будет приведено в следующей главе данной работы.

3.4 Паттерн проектирования *MVVM*

Паттерн *MVVM (Model-View-ViewModel)* был представлен Джоном Госсманом в 2005 году как эволюционное развитие классического паттерна *MVC (Model-View-Controller)*. Его появление было обусловлено необходимостью создания более гибкой архитектуры для современных графических платформ, использующих декларативные языки разметки интерфейса.

Основная цель *MVVM* заключается в достижении полного разделения ответственности между пользовательским интерфейсом и бизнес-логикой приложения. В отличие от более ранних подходов, *MVVM* позволяет дизайнерам и разработчикам логики работать параллельно, не мешая друг другу, так как связь между ними осуществляется через слабую зависимость – механизм привязки данных.

Основными компонентами данного паттерна, как следует из его названия, являются *Model*, *View* и *ViewModel*.

Модель (*Model*) – это фундамент приложения, представляющий собой совокупность данных и бизнес-логики. В данной системе модель не ограничивается простым описанием структур, но также включает в себя сервисы и репозитории. Модель абсолютно ничего не знает о существовании интерфейса, кнопок или окон. Она оперирует чистыми правилами: как рассчитать остаток по кредиту, как проверить корректность даты транзакции и как сохранить эти данные в базу. Благодаря такой изоляции, бизнес-логику можно использовать повторно или полностью изменить способ хранения данных, не затрагивая визуальную часть приложения.

Представление (*View*) – это исключительно визуальный слой, отвечающий за взаимодействие с пользователем. В контексте *MAUI* или аналогичных технологий, представление описывается на языке разметки (*XAML*) и определяет расположение текстовых полей, кнопок, графиков и списков. Главное правило этого слоя — отсутствие логики обработки данных в «коде позади» (*code-behind*). Представление лишь наблюдает за свойствами *ViewModel*. Это позволяет легко менять дизайн приложения, перекрашивать

элементы или менять их расположение, не рискуя сломать логику финансовых расчетов.

Модель представления (*ViewModel*) — это наиболее сложный и ответственный компонент, выступающий интеллектуальным посредником. Её задача — адаптировать данные из Модели под нужды конкретного окна. Например, если во View баланс счета хранится как число, то *ViewModel* преобразует его в отформатированную строку с символом валюты и цветом (красный для долга, зеленый для прибыли). *ViewModel* хранит в себе состояние интерфейса, например, какой элемент в списке сейчас выбран или активно ли поле ввода. Важно, что *ViewModel* не имеет прямых ссылок на элементы, что делает её идеально пригодной для автоматизированного тестирования.

Важной частью паттерна *MVVM* является механизм привязки данных (*Data Binding*). Это технология, позволяющая установить автоматическую связь между свойствами объектов бизнес-логики (*ViewModel*) и элементами пользовательского интерфейса (*View*). Благодаря этому разработчику не нужно вручную прописывать код для обновления каждого текстового поля при изменении данных; система делает это автоматически.

В рамках *MVVM* привязка данных выполняет роль моста, по которому информация перемещается между слоями. Этот механизм классифицируется по направлению потока данных:

- односторонняя привязка (*One-Way*): данные передаются исключительно от источника (*ViewModel*) к приемнику (*View*). Это идеально подходит для отображения информации, которая не предполагает редактирования пользователем, например, текущего общего баланса или истории транзакций;

- двусторонняя привязка (*Two-Way*): обеспечивает синхронизацию в обоих направлениях. Изменения во *ViewModel* отражаются в интерфейсе, а ввод пользователя в элементы управления, например, сумма новой транзакции в поле ввода автоматически записывается обратно в свойства *ViewModel*. Это критически важно для форм создания сущностей и редактирования профиля;

- привязка от приемника к источнику (*One-Way-to-Source*): используется реже, в основном для передачи состояния элементов интерфейса, например, позиции курсора или фокуса ввода обратно в логику приложения.

Для корректной работы этого механизма объект-источник (*ViewModel*) должен обладать способностью «оповещать» систему об изменениях. Это реализуется через стандартный интерфейс *INotifyPropertyChanged*. Когда значение свойства меняется, объект генерирует событие, на которое подписан механизм привязки. Получив сигнал, интерфейс понимает, какой именно элемент управления нужно перерисовать.

Привязка данных не ограничивается только текстом. Она позволяет управлять состоянием кнопок (активна/заблокирована), цветом элементов (например, выделение отрицательного баланса красным) и содержимым списков.

Таким образом, можно выделить следующие весомые преимущества использования паттерна *MVVM*:

- глубокое разделение ответственности: разделение приложения на три независимых слоя гарантирует, что изменения в визуальной части минимально влияют на бизнес-логику, позволяя дизайнерам и программистам работать над своими модулями параллельно без возникновения конфликтов;

- высокая тестируемость: поскольку модель представления является обычным классом и не зависит от визуальных элементов вроде кнопок или окон, всю финансовую логику, включая валидацию ввода и расчеты, можно покрыть автоматизированными *Unit*-тестами без запуска графического интерфейса;

- упрощение поддержки и расширяемости системы: благодаря слабой связанности компонентов через механизмы привязки данных, замена способа визуализации, например, переход от текстовых таблиц к графическим диаграммам, происходит исключительно на уровне представления и не требует переписывания ядра приложения;

- привязка данных: автоматическая синхронизация данных между источником и приемником избавляет разработчика от написания избыточного обслуживающего кода, что значительно снижает вероятность ошибки при обновлении полей ввода или отображении информации;

- эффективное управление состоянием: модель представления всегда аккумулирует актуальное состояние экрана в виде данных, что упрощает процессы сохранения, восстановления и передачи информации между различными частями программы без риска потери контекста при переключении окон.

4 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

4.1 Используемые технологии

Процесс проектирования системы учета личных финансов базируется на выборе современных инструментов, обеспечивающих надежность хранения данных и высокую скорость разработки. В качестве технологического фундамента был выбран стек технологий на базе платформы .NET, который включает в себя следующие компоненты:

- *.NET MAUI (Multi-platform App UI)* является эволюционным развитием *Xamarin.Forms* и применяется для создания визуального интерфейса приложения. Использование *MAUI* позволяет реализовать декларативный подход к построению интерфейса с помощью языка *XAML*, что сочетается с паттерном *MVVM* и механизмами привязки данных, описанными ранее;

- в качестве объектно-реляционного отображения (*ORM*) используется *Entity Framework Core* – современный и легковесный инструмент, который выступает посредником между программным кодом на языке *C#* и базой данных. Главная особенность этой технологии заключается в подходе *Code-First*, который позволяет описывать структуру базы данных через обычные классы. *Entity Framework Core* берет на себя автоматическую генерацию *SQL*-запросов, управление миграциями и отслеживание состояний объектов, что значительно упрощает реализацию приложения и гарантирует типизацию данных на всех этапах работы;

- роль системы управления базами данных (СУБД) играет *MySQL* – одна из самых популярных и надежных реляционных баз данных с открытым исходным кодом. Выбор данной СУБД обусловлен её высокой производительностью при работе с большим количеством транзакционных данных, поддержкой сложных связей между таблицами и гарантией целостности информации. Использование *MySQL* в сочетании с *Entity Framework Core* обеспечивает стабильное хранение финансовых записей и быстрый доступ к аналитической информации.

Выбранный стек технологий позволяет создать масштабируемое приложение с четким разделением ответственности. Интеграция этих инструментов обеспечивает прозрачную передачу данных от пользовательского интерфейса на базе *MAUI* через логические фильтры *ORM*-системы в надежное хранилище *MySQL*, сохраняя при этом гибкость архитектуры для дальнейшего расширения функционала.

4.2 Проектирование базы данных

На рисунке 4.1 представлена диаграмма созданной базы данных, на которой отражены все сущности системы, их атрибуты и установленные между ними логические связи.

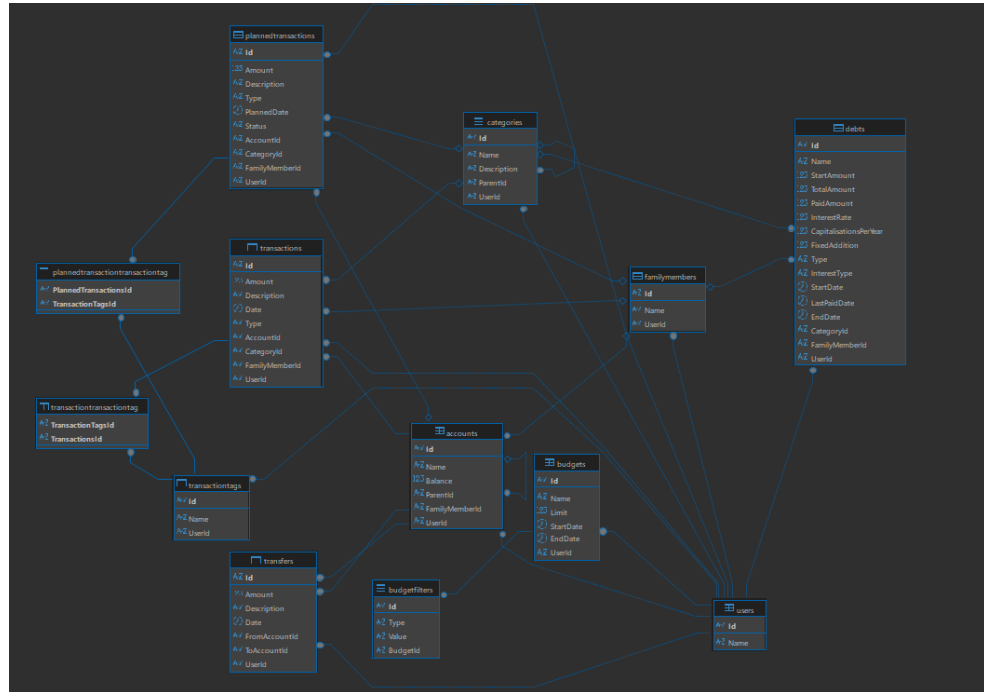


Рисунок 4.1 – Диаграмма базы данных

5 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – СПб. : Питер, 2021. – 352 с
- [2] Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – СПб. : Питер, 2001. – 368 с.
- [3] Васильев, А. Н. Программирование на С# для начинающих. Особенности языка / А. Н. Васильев. – М. : Эксмо, 2019. – 528 с.
- [4] Прайс, М. С# 10 и .NET 6. Современная кроссплатформенная разработка / М. Прайс. – СПб. : Питер, 2023. – 848 с.